Gitlab CI/CD Guide

Jens Bouma
Nicolas Llop Prieto
Husna Keskin
Ben Wierenga
Matz Leeflang

Version Control

Version	Date	Description
0.1	19-12-2024	Initialize document
1	7-1-2025	Document ready

Table of contents

1. Introduction 1
2. Gitlab CI/CD Concepts 2
3. Pipeline Architecture 4
   3.1. Overview of the pipeline structure 4
   3.2. Key stages in our Pipeline 4
   3.3. Dependencies and Execution Flow 4
      3.3.1. Sequential Execution of stages 4
      3.3.2. Parallel Job Execution 4
      3.3.3. Pipeline Visualization 5
4. Pipeline Configuration 6
5. Gitlab Runners 8
6. Environment Variables 10
   6.1. Key Features of Environment variables 10
   6.2. Defining variables in Gitlab 10
   6.3. Best Practices for Environment Variables 11
7. Deployment 11
8. Advise 13
9. Conclusion 14

1. Introduction

This document provides a comprehensive overview of how our GitLab CI/CD pipeline is structured and functions. GitLab CI/CD (Continuous Integration/Continuous Deployment) is an essential tool for automating the process of integrating, testing, and deploying our software applications. By using GitLab's CI/CD features, we ensure that code changes are automatically tested, built, and deployed with minimal manual intervention, streamlining development and enhancing the efficiency of our workflows.

In this document, we will walk through the different stages of our pipeline, describe the roles of key components such as GitLab Runners, Docker, and environment variables, and explain how the various jobs are triggered. This guide will provide the necessary insights into how our pipeline operates.

2. Gitlab CI/CD Concepts

To understand the functionality and structure of our GitLab CI/CD pipeline, it is crucial to become familiar with the core concepts that underpin it. This section introduces these concepts, which form the foundation of our automation and deployment workflows.

Continuous Integration (CI)

Continuous Integration (CI) is the practice of automatically integrating code changes from multiple developers into a shared repository. With GitLab CI:

- Every commit and merge triggers automated testing and validation.
- Bugs are identified early, reducing integration issues and ensuring higher-quality code.
- Developers can focus on building features rather than debugging integration problems.

Continuous Deployment (CD)

Continuous Deployment (CD) automates the process of delivering changes to production environments. In our pipeline:

- Successfully tested code is automatically deployed to predefined environments.
- CD eliminates manual intervention, reducing deployment time and errors.
- It ensures a seamless and consistent delivery process, increasing the reliability of releases.

GitLab Runners

GitLab Runners are lightweight agents that execute the jobs defined in the pipeline. Key aspects include:

- **Types of Runners:**
    - *Shared Runners:* Provided by GitLab, shared across multiple projects.
    - *Specific Runners:* Dedicated to individual projects or teams, offering greater control.
- **Execution Environments:** Runners use environments like Docker containers, virtual machines, or even bare-metal systems to execute tasks.
- Runners fetch the pipeline configuration, perform tasks such as testing, building, and deploying, and then report results back to GitLab.

GitLab Pipeline and Its Components

A GitLab pipeline represents the entire sequence of steps that code undergoes from development to deployment. It is composed of:

Jobs

- The smallest unit of work in a pipeline (e.g., running tests, building an application, or deploying).
- Defined in the **.gitlab-ci.yml** file under specific stages.
- Executed in isolation by a GitLab Runner.

Stages

- Logical groupings of jobs that define the pipeline's flow (e.g., *Build*, *Test*, *Deploy*).
- Stages execute sequentially, while jobs within a stage run in parallel by default.
- Stages ensure an orderly progression through the pipeline, from code validation to deployment.

Pipelines

- The complete sequence of stages and jobs triggered by events like code pushes, merge requests, or schedule-based triggers.
- Pipelines provide a visual representation of progress and outcomes in the GitLab interface.

By understanding these concepts, we can better appreciate the design and functionality of our GitLab CI/CD pipeline. These elements work together to enable seamless integration, testing, and deployment, ensuring the reliability and efficiency of our development processes.

3. Pipeline Architecture

Our GitLab CI/CD pipeline is designed to automate the process from code submission to production deployment. This section provides an overview of its architecture and explains the logical flow between different stages.

3.1. Overview of the pipeline structure

The pipeline consists of multiple stages, each with representing a phase within the software development cycle. These stages are configured in the **.gitlab-ci.yml** file and executed in a predefined order.

3.2. Key stages in our Pipeline

- Test stage:
  - Runs automated test to validate functionality of the (new) code.
  - Includes unit- and cypress tests.
  - Fails the pipeline if any of these tests do not complete successfully.
- Build stage:
  - Compiles the code and prepares docker images for the deployment.
  - Ensures that the code can be build and run.
  - Consists of compiling source code, bundling dependencies and dockerising the applications.
- Deploy stage:
  - Deploys the dockerized applications to the production server
  - Uses docker to deploy and maintain the application
  - Ensures the deployment processes are consistent and repeatable

3.3. Dependencies and Execution Flow

3.3.1. Sequential Execution of stages

Stages are executed in a linear sequence, ensuring that each phase has been completed before proceeding to the next stage.

to explain: the build stage must be successfully completed before the pipeline will proceed to the deployment phase.

3.3.2. Parallel Job Execution

Within a single stage it is possible to run jobs parallel to each other which maximizes on the speed and efficiency of said stage.

For instance, it is possible to run the build job of the front-end and back-end simultaneously to safe time.

3.3.3. Pipeline Visualization

Gitlab provides a clear visual representation of the pipeline in which it is possible to see in what stage the pipeline is currently in and what job(s) are being worked on at that moment. Furthermore, Gitlab shows in what phase a single job is in (paused, running, successful and failure) which makes it possible to monitor and debug.

4. Pipeline Configuration

The **.gitlab-ci.yml** file is the core configuration file for defining pipeline behavior. This section explains its key components and how they influence the pipeline's functionality.

Structure of the .gitlab-ci.yml File

The .gitlab-ci.yml file follows a straightforward YAML structure that defines all stages, jobs, and configurations. Key sections include:

Stages

- Declared at the top of the file to define the pipeline's sequential flow.
- Example:

```yaml
stages:
  - build
  - test
  - deploy
```

Jobs

- Each job specifies tasks for a given stage.
- Example:

```yaml
build_job:
  stage: build
  script:
    - echo "Building the application"
```

Variables

- Allow dynamic customization of pipeline behavior.
- Defined at the global, job, or group level.
- Example:

```yaml
variables:
  FRONTEND_TAG: "$CI_COMMIT_SHA-Nextjs-app"
  BACKEND_TAG: "$CI_COMMIT_SHA-Django-app"
  FRONTEND_NAME: Nextjs-app
  BACKEND_NAME: Django-app
```

Rules

- Define when and how jobs run based on conditions.
- Example:

```
rules:
  - if: '$CI_COMMIT_BRANCH == "main"'
```

Or

```
only:
  - main
```

these two function almost the same. Only applies to branches, it specifies in which branch this job will be able to run. With rules you have more options like run it on commit, Merge Request, etc. one thing to note is that **Only** and **Rules can NOT be used together in a single job**

Best Practices

To ensure an efficient and maintainable pipeline configuration, consider these tips:

- **Use Descriptive Names:** Clearly name jobs and stages for readability.
- **Monitor Performance:** Regularly review pipeline execution times and optimize as needed.
- **Version Control for Configurations:** Track changes to the .gitlab-ci.yml file in the repository.

5. Gitlab Runners

GitLab Runners are lightweight agents that execute the jobs defined in the pipeline. These agents are a critical component of the CI/CD workflow, enabling tasks to run across diverse environments. Below, we explain their functionality and types.

Key Features of GitLab Runners

- **Multi-platform Support:** Runners can operate on various platforms, including Linux, Windows, macOS, and Docker.
- **Isolation:** Jobs are executed in isolated environments to prevent conflicts and ensure reproducibility.
- **Integration:** Runners fetch jobs from the GitLab server, execute them, and send results back for pipeline visualization.

Types of Runners

1. **Shared Runners:**
    - Provided by GitLab and shared across multiple projects.
    - Ideal for small teams or projects that do not require customized execution environments.
    - Managed by GitLab, reducing the need for maintenance.
2. **Specific Runners:**
    - Dedicated to individual projects or groups, providing greater control and customization.
    - Configured by the team, allowing for tailored execution environments.
    - Useful for sensitive or resource-intensive projects where isolation is crucial.
3. **Group Runners:**
    - Associated with a group of projects within a GitLab instance.
    - Allow teams to share runners across related projects while maintaining control over resource allocation.

Execution Environments

GitLab Runners can execute jobs in several environments:

- **Docker:**

    - Provides lightweight, containerized environments for running jobs.
    - Simplifies dependency management and ensures consistency across builds.
- **Shell:**
    - Runs jobs directly on the host machine using the shell environment.
    - Useful for projects with straightforward build requirements.
- **Virtual Machines or Bare-metal Servers:**
    - Offers full control over the build environment.
    - Ideal for resource-intensive or highly customized workflows.

Setting Up a GitLab Runner

to setup your own/new GitLab runners I will refer to the documentation that is provided by GitLab: https://docs.gitlab.com/runner/

Best Practices for Runners

- **Tagging:** Use descriptive tags to categorize runners and assign jobs efficiently.
- **Resource Management:** Monitor runner usage and allocate resources to prevent bottlenecks

6. Environment Variables

Environment variables in GitLab CI/CD pipelines provide a flexible way to manage and customize behavior across different stages and jobs. This chapter delves into their importance, usage, and best practices.

6.1. Key Features of Environment variables

- Allows customization of pipeline behavior without modifying the job codes
- For example, how the containerized application is named as we use in the project:

```
FRONTEND_TAG: "$CI_COMMIT_SHA-Nextjs-app"
BACKEND_TAG: "$CI_COMMIT_SHA-Django-app"
FRONTEND_NAME: Nextjs-app
BACKEND_NAME: Django-app
```

  In this you can see the variables we use to name the containers. The "$CI_COMMIT_SHA" in the variables makes it so that the build can be identified with the commit sha code. This way it is possible to easily rollback to an earlier version of the code if necessary.

- Masking & security:
  In GitLab it is also possible to make variables which are masked and/or protected. Masked means that it is not possible to see what the value of that variable is in the job when it is executed, and protected means that this variable can **only** be used on the **protected branches**, which in this project are the **Main** and **Dev** branch.

6.2. Defining variables in Gitlab

In GitLab it is possible to set variables in different ways.

The first one is to just define the variables in the **.gitlab-ci.yml** file like so:

```
FRONTEND_TAG: "$CI_COMMIT_SHA-Nextjs-app"
BACKEND_TAG: "$CI_COMMIT_SHA-Django-app"
FRONTEND_NAME: Nextjs-app
BACKEND_NAME: Django-app
```

The next way to do it is via the GitLab settings.:

- In the project navigate to Settings > CI/CD > Variables.
  here you can make the variables masked and/or protected

6.3. Best Practices for Environment Variables

- Use descriptive names.
  This will make it clear what the variables are used for and increases efficiency when needing to change something.
- Protect sensitive data
  store sensitive data like passwords and API-keys in the GitLab CI/CD settings with masked enabled.

7. Deployment

The way we deploy the application is with the use of docker containers. With the use of docker containers the application has almost no downtime when deploying from the old version to the new version.

But with efficiency and availability to manage risks I would advise to maybe switch to a blue-green deployment method or canary releases, this makes it so that the risk of a not functioning application can easily be switch back to the last working version of the application.

The way we deploy it makes use of a GitLab-runner which runs a shell engine. This runner connects to the production server with a ssh-token and immediately deploys the docker container to the server and starts it:

```
deploy-frontend:
  stage: deploy
  tags:
    - docker-deploy
  before_script:
    - chmod 400 $SSH_KEY
  script:
    - ssh -o StrictHostKeyChecking=no -i $SSH_KEY root@h2853458.stratoserver.net "
      docker login -u $REGISTRY_USER -p $REGISTRY_PASS &&
      docker ps -a --filter "name=$FRONTEND_NAME" -q | xargs -r docker stop | xargs -r docker rm &&
      docker run -d -p 5000:3000 --name $FRONTEND_NAME $IMAGE_NAME:$FRONTEND_TAG"
  only:
    - main
```

```
deploy-backend:
  stage: deploy
  tags:
    - docker-deploy
  before_script:
    - chmod 400 $SSH_KEY
  script:
    - ssh -o StrictHostKeyChecking=no -i $SSH_KEY root@h2853458.stratoserver.net "
      docker login -u $REGISTRY_USER -p $REGISTRY_PASS &&
      docker ps -a --filter "name=$BACKEND_NAME" -q | xargs -r docker stop | xargs -r docker rm &&
      docker run -d -p 8000:8000 --name $BACKEND_NAME $IMAGE_NAME:$BACKEND_TAG &&
      docker images prune -a"
  only:
    - main
```

As you can see, we have two different deploy jobs. This is so that they can be run parallel to each other if there is a second deploy runner available. The deploy-backend job has the line “docker images prune -a” which removes the old application images so that we don’t cluster the server.

8. Advise

In this chapter we will explain what we think are changes that are good to make for the reliability of the project.

- **Increasing number of runners**
  for now each stage has a single runner that can do the jobs. What this means is that the pipeline takes around 10 minutes to fully finish when this could easily be half of that. To do this we advise to create for each stage a second GitLab-runner which can run the other jobs parallel to the first runner for each stage, which essentially almost halves the required time.
- **Keeping Runners and application on separate servers**
  because we didn't have a second server available the runners are on a single server. This does mean that when that server is down the runners are also not responding and can't finish jobs.
- **Increasing quality**
  when we made this pipeline no one in the team had any experience with making pipelines so the learning journey was quite long. Because of that the quality of the pipeline is also not the best possible way it could have been. That's why we advise that if the next team has someone with experience with pipelines can improve

9. Conclusion

The GitLab CI/CD pipeline ensures a smooth approach to integrating, testing and deploying the application. By leveraging key components such as stages, runners, variables and an automated deployment process, the pipeline ensures consistency reliability and faster delivery of the application.